

```
In [23]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
In [24]: df = pd.read_csv('Boston (1).csv')
```

```
In [25]: df.head()
```

```
Out[25]:
```

	CRIM	ZN	INDUS	CHAS	NX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90



```
In [26]: df.info
```

```
Out[26]: <bound method DataFrame.info of
```

	CRIM	ZN	INDUS	CHAS	NX	RM	AG
E	DIS	RAD	TAX \				
0	0.00632	18.0	2.31	0	0.538	6.575	65.2
1	0.02731	0.0	7.07	0	0.469	6.421	78.9
2	0.02729	0.0	7.07	0	0.469	7.185	61.1
3	0.03237	0.0	2.18	0	0.458	6.998	45.8
4	0.06905	0.0	2.18	0	0.458	7.147	54.2
..	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1
502	0.04527	0.0	11.93	0	0.573	6.120	76.7
503	0.06076	0.0	11.93	0	0.573	6.976	91.0
504	0.10959	0.0	11.93	0	0.573	6.794	89.3
505	0.04741	0.0	11.93	0	0.573	6.030	80.8

	PTRATIO	B	LSTAT	MEDV
0	15.3	396.90	4.98	24.0
1	17.8	396.90	9.14	21.6
2	17.8	392.83	4.03	34.7
3	18.7	394.63	2.94	33.4
4	18.7	396.90	5.33	36.2
..	...	...	...	...
501	21.0	391.99	9.67	22.4
502	21.0	396.90	9.08	20.6
503	21.0	396.90	5.64	23.9
504	21.0	393.45	6.48	22.0
505	21.0	396.90	7.88	11.9

[506 rows x 14 columns]>

```
In [27]: df=df.fillna(df.mean())
```

```
In [28]: X=df.drop('MEDV',axis=1)
```

```
In [29]: y=df['MEDV']
```

```
In [30]: X_train,X_test,y_train,y_test=train_test_split(
        X,y,test_size=0.2,random_state=42
    )
```

test_size=0.2 80% training 20% testing

random_state=42

Ensures same random split every run.

```
In [31]: model = LinearRegression()
```

```
In [32]: model.fit(X_train, y_train)
```

```
Out[32]:
```

LinearRegression

Parameters

```
In [33]: y_pred = model.predict(X_test)
```

```
In [34]: comparison = pd.DataFrame({
    'Actual':y_test,
    'Predicted':y_pred
})
print(comparison.head())
```

	Actual	Predicted
173	23.6	28.996724
274	32.4	36.025565
491	13.6	14.816944
72	22.8	25.031979
452	16.1	18.769880

```
In [35]: mse = mean_squared_error(y_test,y_pred)
print(mse)
```

24.291119474973545

Mean Squared Error Formula

$$MSE = 1/n[\sum(y-y_i)^2]$$

Where:

y = actual value yi = predicted value

```
In [ ]: print(model.coef_)
#actually not needed
```

```
[-1.13055924e-01  3.01104641e-02  4.03807204e-02  2.78443820e+00
 -1.72026334e+01  4.43883520e+00 -6.29636221e-03 -1.44786537e+00
  2.62429736e-01 -1.06467863e-02 -9.15456240e-01  1.23513347e-02
 -5.08571424e-01]
```

Coefficients show impact of each feature.

Positive coefficient: Direct relationship

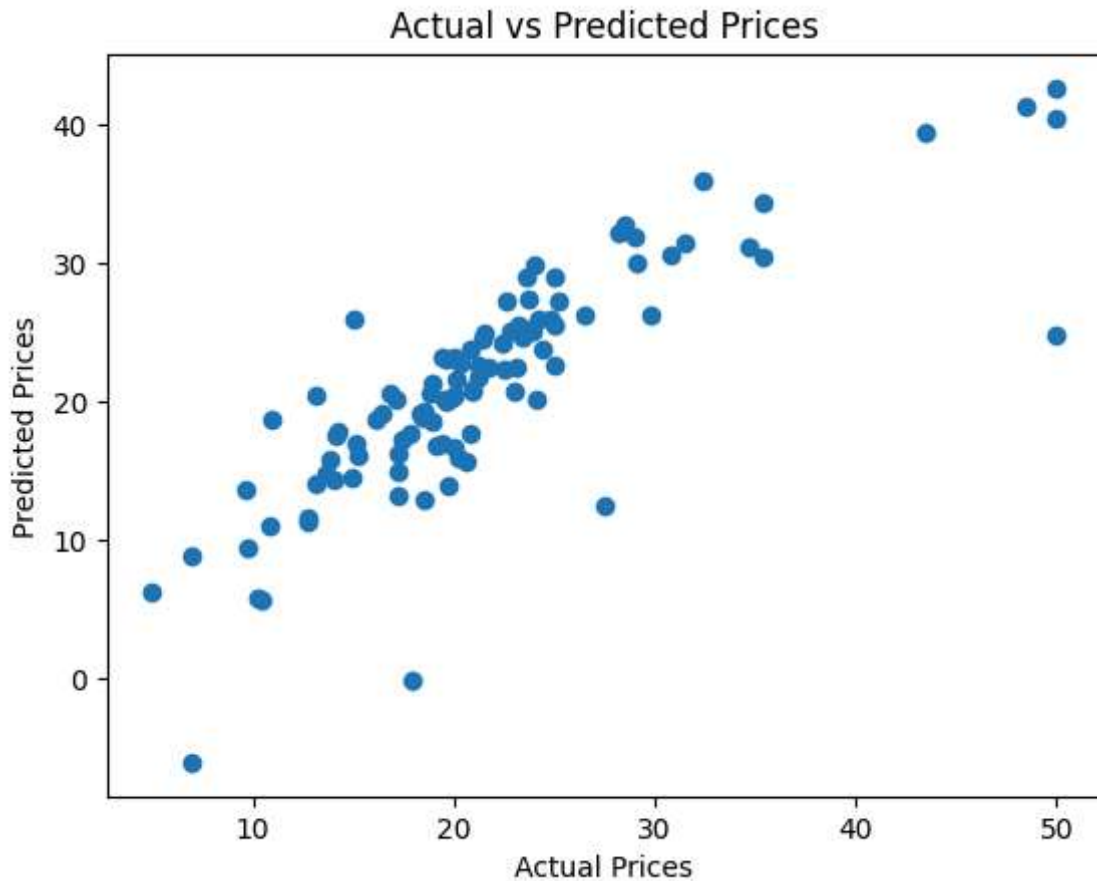
Negative coefficient: Inverse relationship

```
In [37]: plt.scatter(y_test,y_pred)

plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')

plt.title('Actual vs Predicted Prices')

plt.show()
```



DSBDA Practical 4 — Data Analytics I

Linear Regression using Boston Housing Dataset

Dataset Source: [Boston Housing Dataset Kaggle](#)

Aim

To create a Linear Regression model to predict house prices using Boston Housing Dataset.

What is Linear Regression?

Linear Regression is a supervised machine learning algorithm used for:

- Prediction

- Finding relationship between variables

It predicts continuous numerical values.

Example:

- House price prediction
 - Salary prediction
 - Temperature prediction
-

Linear Regression Equation

Simple Linear Regression:

genui★
$$y = mx + c$$

Where:

- y = predicted value
 - x = input feature
 - m = slope
 - c = intercept
-

Multiple Linear Regression

Boston dataset uses multiple features:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where:

- y = house price
 - x_1, x_2 = features
 - b_0 = intercept
 - b_1, b_2 = coefficients
-

About Boston Housing Dataset

Dataset contains:

- Crime rate

- Number of rooms
- Tax rate
- Accessibility
- House prices

Target variable:

MEDV

(Median house value)

Step 1: Import Libraries

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

Explanation

Library	Purpose
pandas	Data handling
numpy	Numerical operations
matplotlib	Visualization
train_test_split	Divide train/test data
LinearRegression	ML model
mean_squared_error	Error calculation

Viva Questions

What is Scikit-learn?

Machine learning library in Python.

Why split dataset?

To test model performance.

Step 2: Load Dataset

Code

```
df = pd.read_csv("HousingData.csv")
```

Display Dataset

Code

```
df.head()
```

Step 3: Check Dataset Information

Code

```
df.info()
```

Explanation

Displays:

- Rows
 - Columns
 - Datatypes
 - Missing values
-

Step 4: Handle Missing Values

Code

```
df = df.fillna(df.mean())
```

Explanation

Replaces missing values with column mean.

Viva Questions

Why handle missing values?

ML models cannot work properly with null values.

Step 5: Define Features and Target

Features (Input Variables)

Code

```
X = df.drop('MEDV', axis=1)
```

Target Variable

Code

```
y = df['MEDV']
```

Explanation

Variable	Meaning
X	Independent variables
y	Dependent variable

Viva Questions

What are independent variables?

Input features used for prediction.

What is dependent variable?

Output value to predict.

Step 6: Split Dataset

Code

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Explanation

`test_size=0.2`

- 80% training
- 20% testing

`random_state=42`

Ensures same random split every run.

Viva Questions

Why training data used?

To train model.

Why testing data used?

To evaluate model.

Step 7: Create Linear Regression Model

Code

```
model = LinearRegression()
```

Explanation

Creates Linear Regression object.

Step 8: Train Model

Code

```
model.fit(X_train, y_train)
```

Explanation

`fit()`

Trains model using training data.

Model learns relationship between:

- Features
 - House prices
-

Viva Questions

What is model training?

Learning patterns from data.

Step 9: Predict House Prices

Code

```
y_pred = model.predict(X_test)
```

Explanation

Predicts prices using testing data.

Step 10: Compare Actual vs Predicted

Code

```
comparison = pd.DataFrame({  
    'Actual': y_test,  
    'Predicted': y_pred  
})  
  
print(comparison.head())
```

Explanation

Compares:

- Actual prices
 - Predicted prices
-

Step 11: Calculate Error

Code

```
mse = mean_squared_error(y_test, y_pred)  
  
print("Mean Squared Error:", mse)
```

Mean Squared Error Formula

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Where:

- y_i = actual value
 - $\hat{y}_i$ = predicted value
-

Explanation

Lower MSE:

- Better model

Higher MSE:

- Poor predictions
-

Step 12: Display Coefficients

Code

```
print(model.coef_)
```

Explanation

Coefficients show impact of each feature.

Positive coefficient:

- Direct relationship

Negative coefficient:

- Inverse relationship
-

Step 13: Display Intercept

Code

```
print(model.intercept_)
```

Explanation

Intercept = value when all features are zero.

Step 14: Visualization

Code

```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")

plt.title("Actual vs Predicted Prices")

plt.show()
```

Explanation

Scatter plot compares:

- Actual values
- Predicted values

Closer points:

- Better predictions
-

Important Theory

What is Supervised Learning?

Learning using:

- Input data

- Correct output labels

Example:

- House price prediction
-

Types of Regression

Type	Use
Linear Regression	Continuous values
Logistic Regression	Classification

Assumptions of Linear Regression

1. Linear relationship
 2. No multicollinearity
 3. Homoscedasticity
 4. Normally distributed errors
-

What is Overfitting?

Model memorizes training data.

Problem:

- Poor test performance
-

What is Underfitting?

Model too simple.

Cannot learn patterns properly.

Difference Between Regression and Classification

Regression	Classification
Predicts continuous values	Predicts categories
Example: price	Example: spam/not spam

Important Viva Questions

Basic Questions

What is Linear Regression?

Algorithm used for predicting continuous values.

What is target variable in Boston dataset?

MEDV (house price).

Why split dataset?

To evaluate model performance.

What does fit() do?

Trains model.

What does predict() do?

Generates predictions.

Intermediate Questions

What is coefficient?

Represents feature importance.

What is intercept?

Prediction when all features are zero.

What is MSE?

Average squared prediction error.

Why squared error used?

Prevents negative cancellation.

Difference between simple and multiple regression?

Simple	Multiple
One feature	Multiple features

Advanced Questions

What if MSE is high?

Model performance is poor.

How to improve regression model?

- More data
 - Feature selection
 - Remove outliers
 - Normalize data
-

What is R^2 score?

Measures goodness of fit.

Formula:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Range:

0 to 1

Higher value = better model.

External May Ask Modifications

Find R² Score

```
print(model.score(X_test, y_test))
```

Use Single Feature

```
X = df[['RM']]
```

Plot Regression Line

```
plt.scatter(X_test['RM'], y_test)
plt.plot(X_test['RM'], y_pred)
plt.show()
```

Predict Single House Price

```
model.predict([[0.1, 18, 2.3, 0, 0.5, 6.5, 65, 4, 1, 296, 15, 390, 4]])
```

Common Errors in Exam

Error	Reason
Shape mismatch	Wrong dimensions
NaN error	Missing values not handled
KeyError	Wrong column name

2-Minute Viva Explanation

"In this practical, we implemented Linear Regression using the Boston Housing dataset to predict house prices. First, we imported required libraries and loaded the dataset using Pandas. We handled missing values using mean imputation. Then we separated independent and dependent variables, split the dataset into training and testing sets, and trained a Linear Regression model using `fit()`. We predicted house prices using `predict()` and evaluated performance using Mean Squared Error. Finally, we visualized actual and predicted prices using a scatter plot."